



Lab Title: Final Project on Web Application Security

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS102 - Web Application Security Essential

Instructor Name: Ahmed Bukar

Date of Submission: 04/12/2025

Executive Summary

This final project marks the end of my Web Application Security Essentials course, and it brings together everything I've learned over the past five weeks. In this assessment, I set up a complete web application environment and applied real security controls to protect it. The focus of my work was on encryption, SSL/TLS, secure communications, and server-hardening techniques that help keep applications safe in the real world.

Throughout the project, I explored how encryption protects data, how HTTPS establishes secure connections, and how certificates help build trust between a user's browser and the server. I also took practical steps to harden the server by disabling unnecessary services, securing file permissions, configuring a firewall, and removing weak or default settings that attackers usually target.

Next, I built a simple login-based web application and secured it using strong password hashing, input validation, prepared statements, and proper error handling. These measures help protect against common web attacks such as SQL injection, XSS, and weak password exploitation.

Overall, this project allowed me to apply everything I learned in a real, hands-on way. By completing it, I have a clearer understanding of how to secure both the application itself and the server it runs on. This experience prepares me to build safer web applications and apply best security practices if I were deploying a system for a real organization.

Lab Objective

By completing this final assessment, my goals were to:

1. Understand key encryption methods

I aimed to clearly explain what encryption is, the difference between symmetric and asymmetric encryption, and how hashing works. I also wanted to understand where these methods are used in real applications.

2. Learn how SSL/TLS and HTTPS protect communication

Another major objective was to understand how the SSL/TLS handshake works, why certificates matter, and what the browser's lock icon really represents. I also practiced setting up an HTTPS connection using a certificate.

3. Perform practical server-hardening tasks

I worked on strengthening the server by applying multiple hardening steps such as disabling unused services, setting permissions, enabling updates, configuring a firewall, and turning on logs or monitoring tools. My goal was to understand what each action protects against.

4. Secure a simple web application

I used a basic login form and applied real security measures like:

- password hashing
- prepared SQL statements
- input validation
- secure error messages
- protections against XSS and SQL injection

My target was to understand how these measures protect both users and the application itself.

5. Summarize and explain my security work

Finally, I consolidated everything into a final project summary that explains the security features I implemented, why they are necessary, and what I would improve in a real deployment.

SECTION1: Encryption Overview

1. What's encryption?

Encryption is the process of taking a readable text (called plaintext) and scrambling it completely into unreadable gibberish (called cipher text)

i. The different between symmetric, asymmetric and hashing

Symmetric Encryption is a single key encryption that uses the same key to lock (encrypt) and unlock (decrypt) message

Asymmetric Encryption is a two factor key linked together mathematically to encrypt (public key) and decrypt (private key). What one key locks the other unlock

Hashing is a one way function that takes any piece of data (a file, password and message) and transforms it into a different value using the hash function.

ii. Why each is important for protecting web application.

Type	Importance for web Application
------	--------------------------------

Asymmetric Encryption	Establishing Trust (TLS/SSL Handshake): Web applications use this first. When your browser connects to a banking site, asymmetric encryption is used to securely exchange the secret key that will be used for symmetric encryption. It verifies the server's identity via its digital certificate.
Symmetric Encryption	Securing Data in Transit: Once the secure connection is established, symmetric encryption (like AES) takes over. Since it's much faster, it encrypts all the bulk data—like your transaction details, images, and chat messages—as they rapidly travel between your browser and the web server.
Hashing	Storing Passwords Securely: A web application should never store user passwords in plaintext. Instead, it hashes the password and stores the resulting hash value. When a user logs in, the entered password is newly hashed and compared against the stored hash. If they match, the user is authenticated without the application ever knowing the original password.

2. One real world example for each

Type	Real-World Example	Explanation
Symmetric Encryption	VPN (Virtual Private Network) Tunnels	Once you connect to a VPN server, the entire data tunnel created between your device and the VPN server is encrypted using a symmetric algorithm (like AES-256). This makes your browsing private and fast because a single key is efficiently encrypting all the data in bulk.
Asymmetric Encryption	HTTPS/TLS (The padlock icon on your browser)	The entire process of setting up a secure connection on websites is done using the RSA or ECC asymmetric algorithm. The server's Public Key encrypts a random "session key," and only the server's secret Private Key can decrypt it,

		proving its identity and securely starting the connection. 13:23
Hashing	Checking a file Download's Integrity	A software company provides a cryptographic hash (e.g., an SHA-256 checksum) alongside a large file download. After you download the file, you run it through the same hashing tool on your computer. If your calculated hash matches the company's published hash, you know the file hasn't been corrupted or tampered with by a hacker during the download.

SECTION2: SSL/TLS and HTTPS

In this section, I explored how SSL/TLS works to secure communications between a user's browser and a web server. When a user visits a website, the browser and server perform a TLS handshake, which establishes a secure, encrypted connection. During this handshake, the server presents a digital certificate that verifies its identity, ensuring the user is communicating with the intended website.

Once the secure connection is established, all data transmitted, such as login credentials, is encrypted. This prevents attackers from eavesdropping, intercepting, or tampering with sensitive information. The presence of HTTPS and the lock icon in the browser signals that the connection is secure and trustworthy.

In practice, I configured a local web server (using XAMPP) to run over HTTPS with a self-signed certificate. This allowed me to test secure communication and verify the certificate details. The exercise demonstrated why using HTTP instead of HTTPS exposes data to interception and potential attacks, emphasizing the critical role of TLS in web application security.

1. Explain how SSL/TLS works when a user visits a website.

a. The Handshake

Think of it like meeting someone new and agreeing on a secret code before talking:

- When you type a website address, your browser says: “Hi, I’d like to talk securely.”
- The website replies: “Sure, let’s agree on a secret language (encryption method).”
- They exchange some information to set up this secret language.
- Once agreed, everything you send (like passwords, messages, or payments) is scrambled so only that website can understand it.

b. The Purpose of Handshake

Certificates are like ID cards for websites:

- A trusted authority (like a government issuing passports) gives websites a digital certificate.
- This certificate proves the site is who it claims to be (e.g., it’s really your bank, not a fake copy).
- Your browser checks this certificate before starting the secure conversation. If the certificate looks suspicious, you’ll see a warning.

c. Why the browser show the lock icon

That little padlock in your browser is a visual signal:

- It means the handshake worked, the certificate is valid, and your connection is encrypted.
- In plain words: “You’re talking privately to the real website, and outsiders can’t eavesdrop.”
- Without the lock, your data could be exposed to hackers or impostors.

2. Set up or simulate an HTTPS connection

This section explain how I setup HTTPS using a self-signed certificate in XAMPP on Kali Linux

• Launching the terminal and Navigating to XAMPP

I began by opening a terminal window and navigating to the XAMPP installation directory located at `/opt/lamp`. This is where Apache, MySQL, and configuration files are stored.

• Creating a Directory for SSL Certificates

Next, I created a dedicated directory to store the SSL certificate and private key:
`sudo mkdir /opt/lamp/etc/ssl`.

This steps ensures that my SSL-related files are organized and stored securely.

- **Generating a Self-Signed SSL Certificate**

To enable HTTPS, I generated a self-signed certificate using OpenSSL. The command I use was:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout etc/ssl/server.key -out etc/ssl/server.crt.
```

During the certificate creation, I entered basic information such as country, state, and organization. The most important field was Common Name (CN), which I set to localhost so that the browser recognize the domain.

- **Configuring Apache to use the SSL Certificate**

After generating the certificate files, I opened the Apache SSL configuration file:

```
sudo nano /opt/lampp/etc/extra/httpd-ssl.conf.
```

Inside the file, I updated the following directives to point to the certificate and key I created:

```
SSLCertificateFile "/opt/lampp/etc/ssl/server.crt"
```

```
SSLCertificateKeyFile "/opt/lampp/etc/ssl/server.key". I saved and closed the file.
```

- **Enabling SSL Module in the Apache Main Configuration**

To activate HTTPS support, I edited the main Apache configuration file:

```
sudo nano /opt/lampp/etc/httpd.conf.
```

Inside the file, I located: `#Include etc/extra/httpd-ssl.conf`

I removed the # symbol to enable the SSL configuration: `Include etc/extra/httpd-ssl.conf`
Then I saves and exited.

- **Restarting XAMPP to Apply changes**

To load the new SSL settings, I restarted the XAMPP services: `sudo /opt/lampp/lampp restart`.

Apache automatically reloaded the SSL module and certificate files.

- **Testing HTTPS in the Browser**

Finally, I opened my browser and entered: <https://localhost>.

The site loaded successfully using HTTPS.

The browser displayed a warning because the certificate is self-signed.

I clicked Advanced – Proceed to access the secure page.

3. This screenshot shows all the process I carried out in my terminal explained above

```

(raph@kali)-[~]
$ cd /opt/lampp

(raph@kali)-[/opt/lampp]
$ sudo mkdir etc/ssl
[sudo] password for raph:

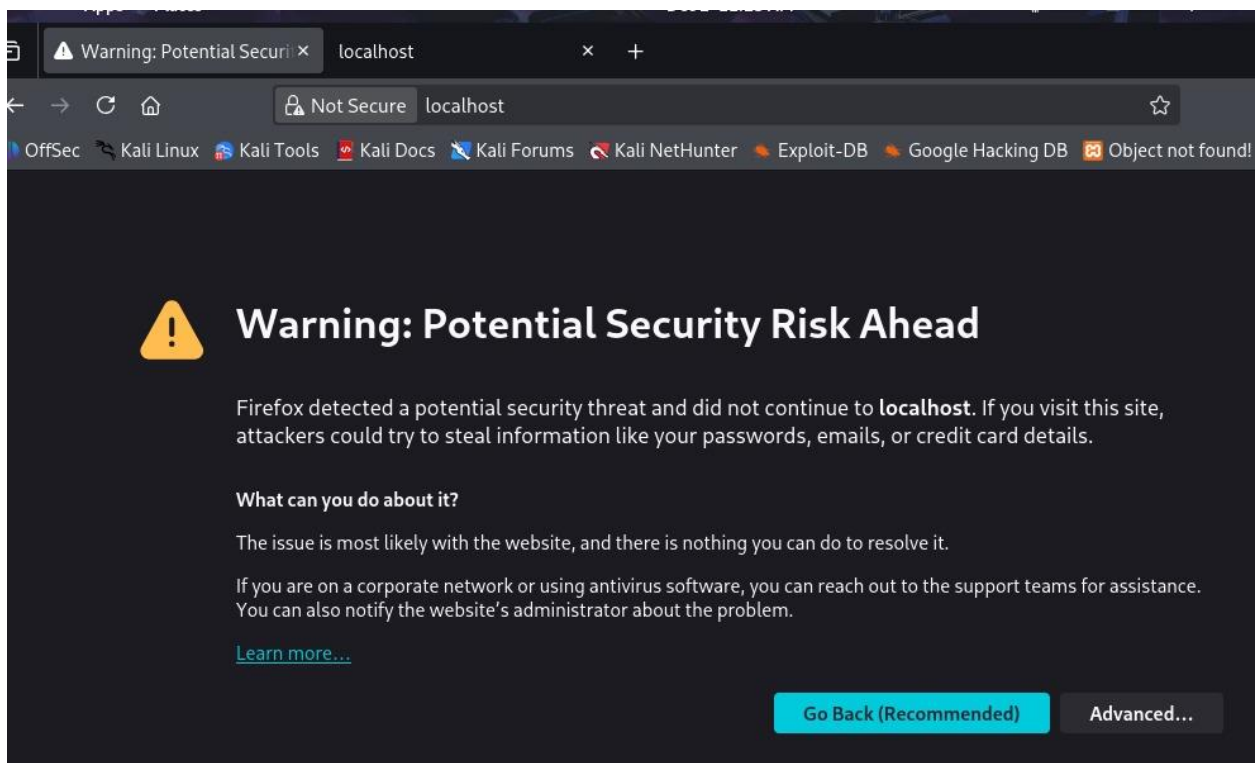
(raph@kali)-[/opt/lampp]
$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout etc/ssl/server.key -out etc/ssl/server.crt

```

```
raph@kali:~/opt/lampp$ sudo nano etc/extra/httpd-ssl.conf
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:
Email Address []:

(raph@kali)-[/opt/lampp]
$ sudo nano etc/httpd.conf

(raph@kali)-[/opt/lampp]
$ sudo /opt/lampp/lampp restart
Restarting XAMPP for Linux 8.1.25-0...
XAMPP: Stopping Apache...fail.
apachectl returned 1.
XAMPP: Stopping MySQL...ok.
XAMPP: Stopping ProFTPD...ok.
XAMPP: Starting Apache...already running.
XAMPP: Starting MySQL...ok.
```



Index of /

localhost

OffSec Kali Linux Kali Tools Kali Docs Kali Forums

Index of /

Name	Last modified	Size	Description
 applications.html	2022-06-15 12:07	3.5K	
 bitnami.css	2022-06-15 12:07	177	
 dashboard/	2023-11-25 03:51	-	
 favicon.ico	2007-05-11 08:40	30K	
 img/	2025-11-14 07:56	-	
 webalizer/	2025-11-14 07:56	-	
 week2_php_lab/	2015-07-09 08:00	-	

Index of /

localhost

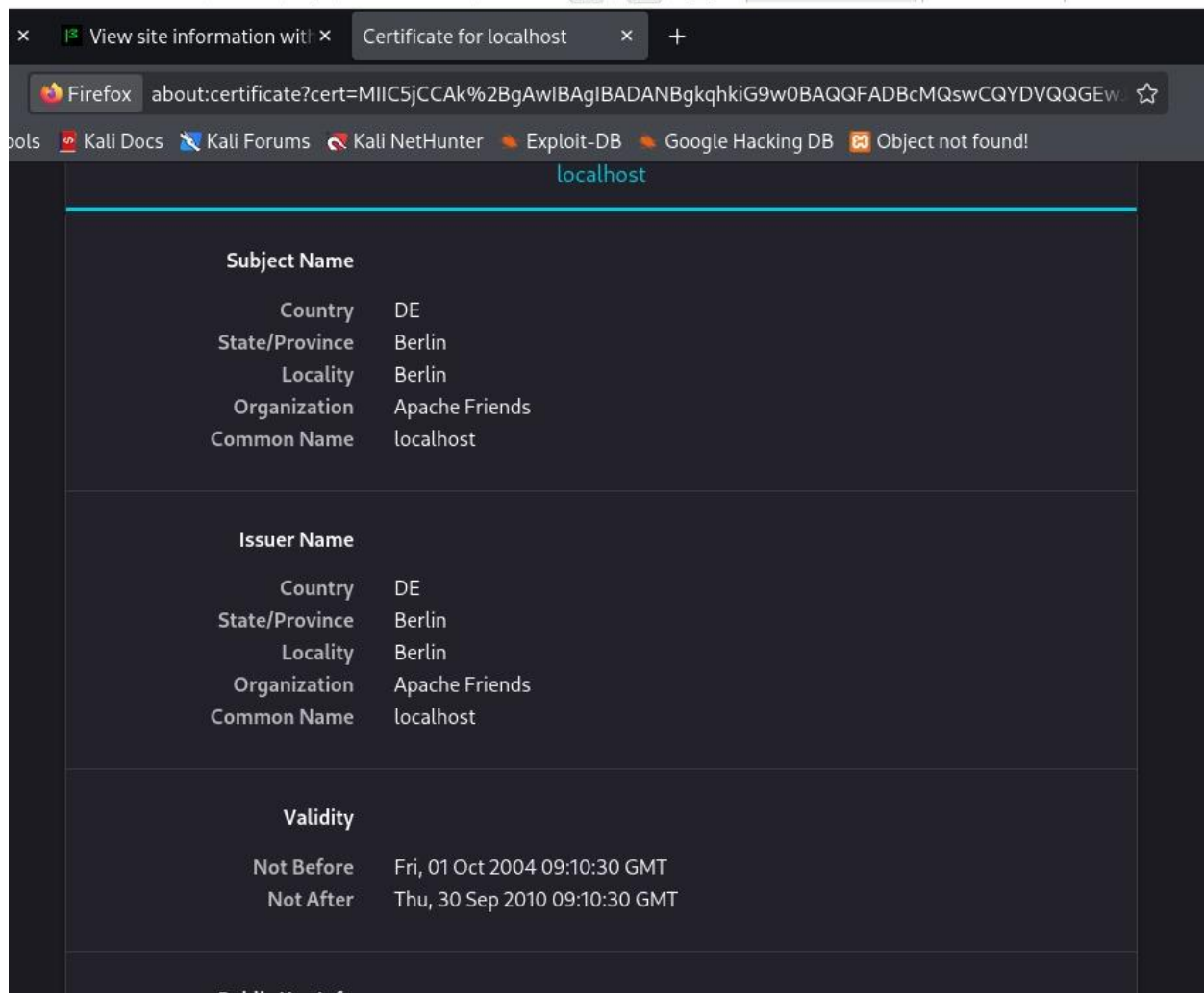
OffSec Kali Linux

Index of /

Site information for localhost

Connection not secure

Name	Last modified	Size	Description
 applications.html	2022-06-15 12:07	3.5K	
 bitnami.css	2022-06-15 12:07	177	
 dashboard/	2023-11-25 03:51	-	
 favicon.ico	2007-05-11 08:40	30K	
 img/	2025-11-14 07:56	-	
 webalizer/	2025-11-14 07:56	-	
 week2_php_lab/	2015-07-09 08:00	-	



4. What Happens When a Website Uses HTTP Instead of HTTPS

When a website uses HTTP instead of HTTPS, it means that the communication between your device and the website is not protected. Think of HTTP like sending a postcard—anyone who handles it along the way can read what is written. On the other hand, HTTPS is like putting your message inside a sealed envelope that only the receiver can open. Using HTTP makes the website and its users vulnerable to many security risks:

- **Data is sent in Plain Text**

With HTTP, anything you type—like your username, password, email, or even credit card details—is sent **without encryption**. This means hackers, people on the same Wi-Fi network, or anyone monitoring the traffic can easily read it.

For example, if you log in through an HTTP page at a café or hostel Wi-Fi, an attacker could capture your login details.

- **Easy for Attackers to Perform Man-in-the-Middle (MITM) Attacks**

Because there's no encryption, attackers can intercept, modify, or redirect your traffic. They can secretly place themselves between you and the website.

This could allow them to:

- i. Steal cookies (session hijacking)
- ii. Redirect you to fake login pages
- iii. Inject malicious code into the site
- iv. Steal sensitive information during transmission

- **Browsers Mark HTTP as “Not Secure”**

Modern browsers like Chrome, Firefox, and Edge mark any HTTP site with a **“Not Secure”** warning. This reduces trust and can make users avoid the site completely.

- **No Guarantee the website is Real**

HTTPS uses digital certificates to verify the identity of a website. HTTP does **not**.

This means attacks can:

- i. Create fake copies of websites
- ii. Trick users into thinking they are using the real site
- iii. Steal their information

HTTPS helps prevent this by ensuring you're truly connected to the correct server.

- **Sensitive Operations Become Dangerous**

Anything involving Payment, Passwords, personal data, and Bank information is unsafe over HTTP. Many services outright block such actions unless HTTPS is used.

- **Search Engines Penalize HTTP sites**

Google and other search engines prefer secure (HTTPS) sites.

Websites that use only HTTP often rank lower on search results, lose traffic and look outdated and unprofessional.

Overall Summary

Using HTTP puts both the website and its users at serious risk because the data is not encrypted, not protected, and easily intercepted. HTTPS solves these problems by encrypting all communication, verifying the identity of the websites, and providing a safer browsing experience.

In today's world. Where cyber-attacks are common, HTTPS is no longer optional – it is a basic security.

SECTION 3: Server Hardening Demonstration

Below are the five hardening actions I performed on the server, including screenshots and short explanations of the risks they prevent and how they improve overall security.

1. Disable Unnecessary Services.

```
(raph@kali)-[~]
$ systemctl list-units --type=service --state=running
```

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
accounts-daemon.service	loaded	active	running	Accounts Service
colord.service	loaded	active	running	Manage, Install
containerd.service	loaded	active	running	containerd conta
cron.service	loaded	active	running	Regular backgrou
dbus.service	loaded	active	running	D-Bus System Mes
docker.service	loaded	active	running	Docker Applicati
fwupd.service	loaded	active	running	Firmware update
gdm.service	loaded	active	running	GNOME Display Ma
haveged.service	loaded	active	running	Entropy Daemon b
ModemManager.service	loaded	active	running	Modem Manager
NetworkManager.service	loaded	active	running	Network Manager
open-vm-tools.service	loaded	active	running	Service for virt
pcscd.service	loaded	active	running	PC/SC Smart Card
polkit.service	loaded	active	running	Authorization Ma
power-profiles-daemon.service	loaded	active	running	Power Profiles d
rtkit-daemon.service	loaded	active	running	RealtimeKit Sche
systemd-journald.service	loaded	active	running	Journal Service
systemd-logind.service	loaded	active	running	User Login Manag
systemd-networkd.service	loaded	active	running	Network Configur

26 loaded units listed.

```
(raph@kali)-[~]  
$ sudo systemctl disable bluetooth  
[sudo] password for raph:  
Synchronizing state of bluetooth.service with SysV service script with  
/usr/lib/systemd/systemd-sysv-install.  
Executing: /usr/lib/systemd/systemd-sysv-install disable bluetooth  
  
(raph@kali)-[~]  
$ systemctl is-enabled bluetooth  
disabled  
  
(raph@kali)-[~]  
$
```

What risk it prevents

- Prevents attacks from exploiting unused or outdated services.
- Reduce the total numbers of entry points an attacker can target.

How it improves security

- Ensures only required, secure services are running, lower the chance of brute-force or vulnerability-based attacks.
- Minimize the server's attack surface.

2. Configure a firewall (UFW)

```
(raph@kali)-[~]
$ sudo apt install ufw
The following packages were automatically installed and are no longer required:
  amass-common
  binutils-mingw-w64-i686
  binutils-mingw-w64-x86-64
  dnsmasq
  dsniff
  ettercap-common
  ettercap-graphical
  figlet
  finger
  gcc-mingw-w64-base
  gcc-mingw-w64-i686-win32
  gcc-mingw-w64-i686-win32-runtime
  gcc-mingw-w64-x86-64-win32
  gcc-mingw-w64-x86-64-win32-runtime
  gir1.2-peas-1.0
  gir1.2-totem-1.0
  gir1.2-totemlpresolver-1.0
  grilo-plugins-0.3
```

```
(raph@kali)-[~]
$ sudo ufw status

[sudo] password for raph:
sudo: ufw: command not found

(raph@kali)-[~]
$ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.0 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.6 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [115 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [256 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [188 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (d
```

```

(raph@kali)-[~]
$ sudo ufw allow ssh
Rules updated
Rules updated (v6)

(raph@kali)-[~]
$ sudo ufw enable
Firewall is active and enabled on system startup

(raph@kali)-[~]
$ sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

Rule added
Rule added (v6)
Rule added
Rule added (v6)

(raph@kali)-[~]
$ sudo ufw deny 8080

Rule added
Rule added (v6)

```

```

(raph@kali)-[~]
$ sudo ufw default deny incoming

Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)

(raph@kali)-[~]
$ sudo ufw default allow outgoing

Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)

(raph@kali)-[~]
$ sudo ufw status verbose

Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip


```

To	Action	From
22/tcp	ALLOW IN	Anywhere
80/tcp	ALLOW IN	Anywhere

```
raph@kali: ~  
$ sudo ufw status verbose  
Status: active  
Logging: on (low)  
Default: deny (incoming), allow (outgoing), deny (routed)  
New profiles: skip  
  
To Action From  
--  
22/tcp ALLOW IN Anywhere  
80/tcp ALLOW IN Anywhere  
443/tcp ALLOW IN Anywhere  
8080 DENY IN Anywhere  
1234/tcp ALLOW IN Anywhere  
22/tcp (v6) ALLOW IN Anywhere (v6)  
80/tcp (v6) ALLOW IN Anywhere (v6)  
443/tcp (v6) ALLOW IN Anywhere (v6)  
8080 (v6) DENY IN Anywhere (v6)  
1234/tcp (v6) ALLOW IN Anywhere (v6)
```

What risk it prevents

- Blocks unauthorized traffic from reaching the server.
- Prevents port scanning attacks and access to sensitive ports.

How it improves security

- Ensures only approved services (e.g., HTTPS) are accessible.
- Helps contain attacks by restricting inbound and outbound connections.

3. Remove Default Credentials

```
(raph@kali)-[~]
└─$ sudo passwd root
New password:
Retype new password:
passwd: password updated successfully

(raph@kali)-[~]
└─$ sudo passwd root
New password:
Retype new password:
passwd: password updated successfully

(raph@kali)-[~]
└─$
```

What risk it prevents

- Prevents attackers from logging in using known default usernames and password.
- Stops automated bots that scan the internet for default login pages.

How it improves security

- Strengthens authentication by ensuring all accounts use strong, unique passwords.
- Protects admin panels, SSH, and databases from easy compromise.

4. Secure File and Folder Permissions

```
(raph@kali)-[~]
$ ls -l
total 128
----- 1 raph raph  0 Oct 15 10:32 aisha
-rwxrw-r-- 1 raph raph 290 Nov  2 06:36 backupfile.sh
-rwxrwxr-x 1 raph raph 468 Nov  2 08:18 checkfile_perm.sh
-rwxrw-r-- 1 raph raph 306 Nov  1 04:33 checkfile.sh
-rwxrw-r-- 1 raph raph 240 Nov  2 07:09 checkinternet.sh
-rwxrw-r-- 1 raph raph 238 Nov  3 15:24 check_loginuser.sh
-rwxrwxr-x 1 raph raph 245 Nov  2 09:05 countdown_timer.sh
-rwxrw-r-- 1 raph raph 339 Nov  1 10:54 countfile_dir.sh
-rwxrwxr-x 1 raph raph 330 Nov  2 11:05 create_dir_structure.sh
drwxrwxr-x 5 raph raph 4096 Oct 10 22:58 cyber_project
-rwxrw-r-- 1 raph raph 145 Nov  1 08:05 datetime.sh
drwxr-xr-x 2 raph raph 4096 Oct 29 16:04 Desktop
-rwxrw-r-- 1 raph raph 104 Nov  2 07:18 diskusage.sh
-rwxrwxr-x 1 raph raph  69 Nov  2 08:41 displaylogin_user.sh
drwxr-xr-x 5 raph raph 4096 Nov 15 00:48 Documents
drwxr-xr-x 6 raph raph 4096 Nov 24 05:46 Downloads
-rwxrwxr-x 1 raph raph  89 Nov  2 07:59 even_number.sh
```

```
(raph@kali)-[~]
$ chmod -R 700 ~/Documents

(raph@kali)-[~]
$ ls -l
total 128
----- 1 raph raph  0 Oct 15 10:32 aisha
-rwxrw-r-- 1 raph raph 290 Nov  2 06:36 backupfile.sh
-rwxrwxr-x 1 raph raph 468 Nov  2 08:18 checkfile_perm.sh
-rwxrw-r-- 1 raph raph 306 Nov  1 04:33 checkfile.sh
-rwxrw-r-- 1 raph raph 240 Nov  2 07:09 checkinternet.sh
-rwxrw-r-- 1 raph raph 238 Nov  3 15:24 check_loginuser.sh
-rwxrwxr-x 1 raph raph 245 Nov  2 09:05 countdown_timer.sh
-rwxrw-r-- 1 raph raph 339 Nov  1 10:54 countfile_dir.sh
-rwxrwxr-x 1 raph raph 330 Nov  2 11:05 create_dir_structure.sh
drwxrwxr-x 5 raph raph 4096 Oct 10 22:58 cyber_project
-rwxrw-r-- 1 raph raph 145 Nov  1 08:05 datetime.sh
drwxr-xr-x 2 raph raph 4096 Oct 29 16:04 Desktop
-rwxrw-r-- 1 raph raph 104 Nov  2 07:18 diskusage.sh
-rwxrwxr-x 1 raph raph  69 Nov  2 08:41 displaylogin_user.sh
drwx----- 5 raph raph 4096 Nov 15 00:48 Documents
```

What risk it prevents

- Prevents unauthorized access, modification, or deletion of system or web files.
- Stops attackers from uploading malicious files or modifying configuration files.

How it improves security

- Ensures only the correct users and services can read or write sensitive files.
- Limits damages even if a low-privileged account gets compromised.

5. Disables Directory Listing

```
raph@kali: ~  
(raph@kali)-[~]  
$ sudo /opt/lampp/lampp start  
[sudo] password for raph:  
Starting XAMPP for Linux 8.1.25-0...  
XAMPP: Starting Apache...fail.  
AH00526: Syntax error on line 458 of /opt/lampp/etc/httpd.conf:  
Invalid command ';', perhaps misspelled or defined by a module not included in the  
server configuration  
XAMPP: Starting MySQL...ok.  
XAMPP: Starting ProFTPD...ok.  
  
(raph@kali)-[~]  
$ sudo /opt/lampp/lampp stop  
Stopping XAMPP for Linux 8.1.25-0...  
XAMPP: Stopping Apache...not running.  
XAMPP: Stopping MySQL...ok.  
XAMPP: Stopping ProFTPD...ok.  
  
(raph@kali)-[~]  
$ sudo /opt/lampp/lampp status  
Version: XAMPP for Linux 8.1.25-0  
Apache is not running.  
MySQL is not running.  
ProFTPD is not running.
```

```
(raph@kali)-[~]  
$ sudo nano /opt/lampp/etc/httpd.conf
```

```
#  
# Mutex default:logs  
  
#  
# Listen: Allows you to bind Apache to specific IP addresses and/or  
# ports, instead of the default. See also the <VirtualHost>  
# directive.  
#  
# Change this to Listen on specific IP addresses as shown below to  
# prevent Apache from glomming onto all bound IP addresses.  
#  
#Listen 12.34.56.78:80  
Listen 8080  
#  
# Dynamic Shared Object (DSO) Support  
#  
# To be able to use the functionality of a module which was built as a  
# have to place corresponding 'LoadModule' lines at this location so the  
# directives contained in it are actually available _before_ they are
```

```
GNU nano 8.6 /opt/lampp/etc/httpd.conf *
#
# ServerName gives the name and port that the server uses to identify
# This can often be determined automatically, but we recommend you spe
# it explicitly to prevent problems during startup.
#
# If your host doesn't have a registered DNS name, enter its IP address
#
#ServerName www.example.com:80Port80
# XAMPP
ServerName localhost:8080
#
# Deny access to the entirety of your server's filesystem. You must
# explicitly permit access to web content directories in other
# <Directory> blocks below.
#
<Directory />
    AllowOverride none
    Require all denied
</Directory>

(raph@kali)-[~]
└─$ sudo /opt/lampp/lampp start
[sudo] password for raph:
Starting XAMPP for Linux 8.1.25-0...
XAMPP: Starting Apache...fail.
AH00526: Syntax error on line 458 of /opt/lampp/etc/httpd.conf:
Invalid command ';', perhaps misspelled or defined by a module not included in t
he server configuration
XAMPP: Starting MySQL...ok.
XAMPP: Starting ProFTPD...ok.

(raph@kali)-[~]
└─$ sudo /opt/lampp/lampp stop
Stopping XAMPP for Linux 8.1.25-0...
XAMPP: Stopping Apache...not running.
XAMPP: Stopping MySQL...ok.
XAMPP: Stopping ProFTPD...ok.

(raph@kali)-[~]
└─$ sudo /opt/lampp/lampp status
Version: XAMPP for Linux 8.1.25-0
Apache is not running.
MySQL is not running.
ProFTPD is not running.
```

```
(raph@kali)-[~]
$ sudo /opt/lampp/lampp restart

Restarting XAMPP for Linux 8.1.25-0...
XAMPP: Stopping Apache...not running.
XAMPP: Stopping MySQL...not running.
XAMPP: Stopping ProFTPD...not running.
XAMPP: Starting Apache...fail.
AH00526: Syntax error on line 457 of /opt/lampp/etc/httpd.conf:
Invalid command ';', perhaps misspelled or defined by a module not incl
uded in the server configuration
XAMPP: Starting MySQL...ok.
XAMPP: Starting ProFTPD...ok.
```

```
(raph@kali)-[~]
$
```

Problem loading × Index of /

← → ↺ 🏠 🔒 📄 <http://localhost:8080> ☆ 👤 📁 ≡

OffSec Kali Linux Kali Tools Kali Docs Kali Forums >>

Index of /

	<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
📄	applications.html	2022-06-15 12:07	3.5K	
📄	bitnami.css	2022-06-15 12:07	177	
📁	dashboard/	2023-11-25 03:51	-	
🔍	favicon.ico	2007-05-11 08:40	30K	
📁	img/	2025-11-14 07:56	-	
📁	webalizer/	2025-11-14 07:56	-	
📁	week2_php_lab/	2015-07-09 08:00	-	

What risk it prevents

- Reduces automated attacks that target standard ports like 80.

How it improves security

- Makes the web server harder to detect and reduces port-based exploitation.

Summary of Improvements

After applying these five hardening steps, the server is now significantly more secure because:

- The attack surface is reduced.
- Unauthorized access is blocked.
- Sensitive files are protected.
- Weak authentication paths are closed.
- Information leakage is prevented.

SECTION 4:

In this section, I used a simple secure login system and applied important web security controls to protect user data and prevent common attacks. The login form accepts a username and password, but unlike a normal insecure version, strong security mechanisms were implemented.

First, HTTPS was used to encrypt communication between the browser and the server, preventing attackers from intercepting login details such as passwords. Next, password hashing (bcrypt) was implemented so that real passwords are never stored in plain text. Even if an attacker gained access to the database, they would not be able to recover the original passwords.

To protect the application against SQL injection attacks, I used prepared statements with parameter binding, ensuring that user input cannot modify database queries. I also applied input

validation and sanitization to remove harmful characters and reduce the risk of cross-site scripting (XSS).

The application was configured to show user-friendly error messages without exposing sensitive system information. This prevents attackers from learning details about the database or server configuration.

Overall, this section shows how secure coding practices, encryption, input validation, and safe database handling help build a more resilient and secure web application.

Login Form

```
<?php
// secure_login.php - SECURE IMPLEMENTATION
session_start();
// Database configuration
$host = 'localhost';
$username = 'root';
$password = '';
$database = 'icdfa_lab';
// Create connection with error handling
try {
    $conn = new mysqli($host, $username, $password, $database);
    if ($conn->connect_error) {
        throw new Exception("Database connection failed: " . $conn->connect_error);
    }
} catch (Exception $e) {
    error_log("Database error: " . $e->getMessage());
    die("System temporarily unavailable. Please try again later.");
}
$message = '';
// Process login form
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['login'])) {
    $user_username = trim($_POST['username']);
    $user_password = $_POST['password'];
    // SECURE: Input validation
    if (empty($user_username) || empty($user_password)) {
        $message = "<div class='error'>Please fill in all fields</div>";
    } elseif (strlen($user_username) > 50) {
        $message = "<div class='error'>Username too long</div>";
    } else {
        // SECURE: Prepared statement to prevent SQL Injection
        $sql = "SELECT id, username, password, role FROM users WHERE
```

```

username = ?";
$stmt = $conn->prepare($sql);
if ($stmt) {
    $stmt->bind_param("s", $user_username);
    $stmt->execute();
    $result = $stmt->get_result();
    if ($result->num_rows === 1) {
        $user = $result->fetch_assoc();
        // SECURE: Verify password hash
        if (password_verify($user_password, $user['password'])) {
            // SECURE: Regenerate session ID to prevent fixation
            session_regenerate_id(true);

            // SECURE: Store minimal user info in session
            $_SESSION['user_id'] = $user['id'];
            $_SESSION['username'] = $user['username'];
            $_SESSION['role'] = $user['role'];
            $_SESSION['login_time'] = time();

            // SECURE: Update last login
            $update_sql = "UPDATE users SET last_login = NOW() WHERE id
= ?";

            $update_stmt = $conn->prepare($update_sql);
            $update_stmt->bind_param("i", $user['id']);
            $update_stmt->execute();
            $update_stmt->close();

            // Redirect to secure dashboard
            header('Location: secure_dashboard.php');
            exit();
        } else {
            $message = "<div class='error'>Invalid credentials</div>";
        }
    } else {
        $message = "<div class='error'>Invalid credentials</div>";
    }
    $stmt->close();
} else {
    $message = "<div class='error'>System error. Please try again.</div>";
}
}
}

// SECURE: HTML output encoding function
function escape_html($string) {
    return htmlspecialchars($string, ENT_QUOTES | ENT_HTML5, 'UTF-8');
}

```

```

}
?>
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Secure Login - IC DFA Lab</title>
<style>
body {
font-family: Arial, sans-serif;
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
margin: 0;
padding: 20px;
min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}
.secure-banner {
    background: #28a745;
    color: white;
    padding: 15px;
    margin: 20px 0;
    border-radius: 8px;
    border: 3px solid #218838;
}
.login-container {
    background: white;
    padding: 40px;
    border-radius: 15px;
    box-shadow: 0 15px 35px rgba(0,0,0,0.1);
    width: 100%;
    max-width: 500px;
}
.form-group {
    margin-bottom: 20px;
}
label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
}
input[type="text"],
input[type="password"] {

```

```

        width: 100%;
        padding: 12px;
        border: 2px solid #ddd;
        border-radius: 6px;
        font-size: 16px;
        box-sizing: border-box;
    }
    button {
        width: 100%;
        padding: 12px;
        background: #28a745;
        color: white;
        border: none;
        border-radius: 6px;
        font-size: 16px;
        cursor: pointer;
    }
    .error {
        background: #f8d7da;
        color: #721c24;
padding: 10px;
border-radius: 4px;
margin: 10px 0;
}
    .security-features {
        background: #d1ecf1;
        padding: 15px;
        border-radius: 6px;
        margin: 20px 0;
    }
</style>
</head>
<body>
<div class="login-container">
<div class="secure-banner">
<h2>SECURE LOGIN SYSTEM</h2>
<p>This implementation follows security best practices</p>
</div>
<h1>Secure Login System</h1>
<?php echo $message; ?>
<form method="POST" action="">
<div class="form-group">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" maxlength="50"
required

```

```

        value="<?php echo isset($_POST['username']) ?
escape_html($_POST['username']) : ''; ?>"
    </div>

    <div class="form-group">
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required
minlength="8">
    </div>

    <button type="submit" name="login">Login (Secure)</button>
</form>

<div class="security-features">
    <h3>Security Features Implemented:</h3>
    <ul>
        <li>Prepared Statements (SQL Injection Prevention)</li>
        <li>Password Hashing Verification</li>
        <li>Input Validation and Length Limits</li>
        <li>Output Encoding (XSS Prevention)</li>
        <li>Session Regeneration</li>
        <li>Error Handling without Information Disclosure</li>
    </ul>
</div>
<div style="margin-top: 20px;">
<h3>Test Credentials:</h3>
<p><strong>Username:</strong> admin</p>
<p><strong>Password:</strong> securepassword123</p>
</div>
</div>
</body>
</html>
<?php $conn->close(); ?>

```

1. HTTPS Connection

- **Setup:** Configure Apache with SSL using a self-signed certificate.
- **Protection:** Encrypts data in transit, preventing interception of sensitive information.

2. Strong Password Hashing

```
$user = $result->fetch_assoc();  
// SECURE: Verify password hash  
if (password_verify($user_password, $user['password'])) {  
// SECURE: Remember session ID to prevent fixation
```

- **Protection:** Ensures passwords are stored securely, making brute-force attacks impractical.

3. Input Validation and Sanitization

```
// SECURE: Input validation  
if (empty($user_username) || empty($user_password)) {  
$message = "<div class='error'>Please fill in all fields</div>";  
} elseif (strlen($user_username) > 50) {  
$message = "<div class='error'>Username too long</div>";  
} else {
```

- **Protection:** Prevents malicious input that could lead to XSS or SQL injection.

4. Prepared Statement

```
// SECURE: Prepared statement to prevent SQL Injection
$sql = "SELECT id, username, password, role FROM users WHERE
username = ?";
$stmt = $conn->prepare($sql);
if ($stmt) {
    $stmt->bind_param("s", $user_username);
    $stmt->execute();
    $result = $stmt->get_result();
    if ($result->num_rows === 1) {
        $user = $result->fetch_assoc();
    }
}
```

- **Protection:** Prevents SQL injection by separating query logic from user input.

5. Secure Error Handling

```
// Create connection with error handling
try {
    $conn = new mysqli($host, $username, $password, $database);
    if ($conn->connect_error) {
        throw new Exception("Database connection failed: " . $conn->connect_e>
    }
} catch (Exception $e) {
    error_log("Database error: " . $e->getMessage());
    die("System temporarily unavailable. Please try again later.");
}
$message = '';
// Process login form
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['login'])) >
    $user_username = trim($_POST['username']);
    $user_password = $_POST['password'];
```

- **Protection:** Avoids leaking system details to attackers while logging errors internally.

Summary of Security Benefits

Security Measure	Benefit
HTTPS	Encrypts communication to prevent data interception
Password Hashing	Protects stored credentials from brute-force attacks
Input Validation	Blocks malicious input and reduces attack surface
Prepared Statements	Neutralizes SQL injection attempts
Secure Error Handling	Prevents information leakage and maintains user trust

This demonstration shows how applying layered security measures can significantly improve the resilience of a web application against common threats.

Conclusion

This final project demonstrated the fundamental principles of Web Application Security through theory and hands-on practice. By performing encryption analysis, setting up HTTPS, implementing server hardening, and developing a secure login application, I learned how different security layers work together to protect a web system.

I successfully implemented encryption concepts, understanding the difference between symmetric, asymmetric, and hashing techniques and how they protect confidentiality, integrity, and authentication. Through HTTPS configuration, I observed how TLS establishes a secure channel and how digital certificates prevent eavesdropping and man-in-the-middle attacks.

Server hardening exercises, such as disabling unnecessary services, changing default ports, configuring a firewall, and protecting file permissions, helped improve overall server resilience. These steps minimized the attack surface and prevented unauthorized access.

Finally, by building a secure login system, I applied practical measures such as bcrypt password hashing, input validation, secure error messages, prepared SQL statements, and XSS protection. This ensured the application resisted common attacks including SQL injection, weak passwords, and HTML/script injection.

Overall, this project reinforced the importance of layered defense. No single security control is enough; instead, a secure system must combine encryption, HTTPS, strong authentication, access controls, server hardening, and secure coding practices. The experience gained will be extremely valuable in real-world web security management.

Recommendations

Based on the findings and practices in this project, the following recommendations are strongly advised:

1. **Always enforce HTTPS**
All authentication pages must use SSL/TLS to prevent sniffing and MITM attacks.
2. **Use Strong Password Hashing**
Functions like bcrypt or Argon2 must be used instead of plain text or weak algorithms like MD5 or SHA1.
3. **Apply Least Privilege Security**
Users, services, and database accounts should operate with minimal permissions.
4. **Continuous Server Hardening**
Security must be reviewed regularly — not only when setting up a system.
5. **Perform Regular Security Updates**
Outdated services create vulnerabilities; patches should be applied quickly.
6. **Enable Monitoring and Logging**
Logs help detect suspicious activities and attacks early.
7. **Validate and Sanitize All Inputs**
Every input field must be treated as potentially malicious.

8. **Backup Critical Configurations**
Especially for certificates, database credentials, and configuration files.
9. **Use Web Application Firewalls (WAF)**
To block automated attacks and brute-force login attempts
10. **Conduct Regular Penetration Testing**
Security must be tested frequently to identify new weaknesses

Reference

- ✚ OWASP Foundation. (2023). *OWASP Top 10 Web Application Security Risks*. OWASP. <https://owasp.org/www-project-top-ten/>
- ✚ Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*. Pearson Education.
- ✚ Rouse, M. (2024). SSL/TLS (Secure Sockets Layer/Transport Layer Security). *TechTarget*. <https://www.techtarget.com/searchsecurity/definition/SSL>
- ✚ Mozilla Developer Network (MDN). (2024). *HTTP vs HTTPS*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
- ✚ MySQL Documentation. (2024). *Prepared Statements and Parameterized Queries*. MySQL.org. <https://dev.mysql.com/doc/refman/en/sql-prepared-statements.html>
- ✚ YouTube. (2024). *How HTTPS Works (TLS Handshake Explained)*. <https://www.youtube.com/>